

Documento Técnico: Estructura e Implementación del Frontend — Proyecto HeartCoin

Introducción

Este documento describe la estructura interna de la aplicación cliente de HeartCoin, desarrollada en Flutter, así como las decisiones de implementación adoptadas para organizar sus pantallas, componentes reutilizables y manejo de sesión de usuario. Consolida en un solo documento cinco actividades relacionadas —separación del frontend, reorganización de vistas/componentes/servicios, desarrollo de pantallas principales, implementación de componentes reutilizables e implementación de manejo de sesión— dado que todas describen, desde distintos ángulos, la misma capa del sistema.

Objetivo

Documentar la organización estructural del código del frontend, el catálogo de pantallas y componentes que lo componen, y el mecanismo mediante el cual la aplicación gestiona la sesión activa del usuario.

Alcance

Este documento cubre la estructura de directorios de la aplicación Flutter, el patrón de diseño de sus servicios, el catálogo de pantallas principales por rol de usuario, los componentes de interfaz reutilizables, y el manejo de sesión desde la perspectiva del cliente. No cubre el diseño visual (maquetas), pendiente de recibir material de referencia adicional, ni el detalle de las políticas de seguridad del backend (referido a `ESTRATEGIA_AUTENTICACION_AUTORIZACION.md`).

Metodología

La información se obtuvo mediante inspección directa del código fuente de la aplicación Flutter a lo largo de todo el desarrollo del proyecto, incluyendo la totalidad de las pantallas, servicios y widgets construidos.

Desarrollo del Análisis

1. Separación del Frontend

La aplicación cliente vive en un módulo autocontenido (`lib/`), sin dependencias de código hacia el backend más allá del consumo de su API a través del SDK oficial de Supabase. Esta separación es completa a nivel de código: no existe lógica de negocio del backend replicada en el cliente, ni credenciales privilegiadas embebidas en él (únicamente la `anon key` pública de Supabase).

2. Reorganización de Vistas, Componentes y Servicios

El código del frontend se organiza en cuatro directorios principales:

```
lib/ ■■■ main.dart # Punto de entrada, rutas, tema, configuración global ■■■ screens/ #  
Pantallas de la aplicación, organizadas por rol/dominio ■■■ services/ # Lógica de acceso  
a datos, un archivo por dominio funcional ■■■ widgets/ # Componentes de interfaz  
reutilizables entre pantallas ■■■ theme/ # Paleta de colores oficial de la aplicación
```

Patrón de servicio-singleton: cada dominio funcional cuenta con una clase de servicio dedicada, instanciada como singleton (`ServiceName.instance`), que encapsula la totalidad de las llamadas al backend para ese dominio. Ejemplos: `AuthService`, `WalletService`, `FollowService`, `BlockService`, `CheckinService`, `OriginalAuthService`. Este patrón evita que la lógica de acceso a datos se disperse directamente dentro de los widgets de pantalla, centralizándola en un punto único y fácilmente localizable por dominio.

Patrón de excepciones tipadas por servicio: cada servicio que puede fallar de forma significativa define su propia clase de excepción (`AuthServiceException`, `DeleteAccountException`, `OriginalAuthException`, `MediaUploadException`), permitiendo que la capa de presentación capture errores específicos de dominio en lugar de excepciones genéricas, y muestre mensajes de error adaptados al contexto.

3. Desarrollo de Pantallas Principales del Sistema

Las pantallas se organizan por rol de usuario y por dominio funcional. El sistema soporta **tres roles** (Personal, Organización, Empresa), cada uno con su propio conjunto de pantallas principales:

Rol	Pantallas principales
Personal	Feed (<code>home_people_screen.dart</code>), Explorar, Billetera, Perfil (propio y de terceros), Tus acciones, Certificados, Configuración
Organización	Dashboard, Mis iniciativas, Monitor de votaciones, Comunidad
Empresa	Dashboard, Mis beneficios, Mis servicios

Adicionalmente, existe un conjunto de pantallas transversales de autenticación (login, registro, recuperación de contraseña de tres pasos, verificación de correo) y de configuración de cuenta (editar perfil, cambiar contraseña, notificaciones, privacidad, política de privacidad, usuarios bloqueados, eliminar cuenta), agrupadas bajo el flujo de "Configuración" accesible desde el rol Personal.

Cada pantalla de detalle (iniciativa, votación, servicio, beneficio) se diseñó como un componente único que **adapta su comportamiento según el rol del usuario que la visualiza** —por ejemplo, la pantalla de detalle de un servicio muestra controles de gestión (activar/desactivar, mostrar QR) al dueño empresarial,

y un botón de canje a cualquier otro usuario— evitando la duplicación de pantallas equivalentes por rol.

4. Implementación de Componentes Reutilizables

El directorio `widgets/` concentra los componentes de interfaz compartidos entre múltiples pantallas:

Componente	Propósito	Reutilizado en
<code>FollowButton</code>	Botón "Seguir/Siguiendo" con estado propio	Feed, detalle de iniciativa, perfil de otro usuario
<code>SaveButton</code>	Botón "Guardar" en modo normal y compacto	Tarjetas y pantallas de detalle de iniciativas/votaciones
<code>PostsGrid</code>	Cuadrícula de publicaciones (3 columnas)	Perfil propio y perfil de otro usuario
<code>IniciativaWidgets</code> (tarjetas y chips)	Presentación consistente de iniciativas en listas	Explorar, iniciativas por categoría, iniciativas de ahorro/social

La extracción de `PostsGrid` como componente compartido, en particular, se realizó explícitamente para evitar la duplicación de código entre la pantalla de perfil propio y la de perfil de terceros, ambas construidas sobre la misma estructura visual.

5. Implementación de Manejo de Sesión

El manejo de sesión del lado del cliente descansa íntegramente en el SDK de Supabase, sin un mecanismo de sesión propio adicional:

- La sesión activa (`access_token/refresh_token`) es gestionada internamente por `supabase_flutter`, incluyendo la renovación automática del token antes de su expiración.
- El rol del usuario (Personal/Organización/Empresa) se lee directamente de `user_metadata` de la sesión local, sin requerir una llamada de red adicional en cada verificación.
- El cierre de sesión (`signOut()`) invalida la sesión tanto localmente como en el servidor.
- Las preferencias de interfaz no vinculadas a la cuenta (tamaño de texto, preferencia de ubicación) se persisten de forma independiente mediante `shared_preferences` del dispositivo, sobreviviendo a cierres de la aplicación sin depender de la sesión de Supabase.

Patrón de propagación de estado global: para preferencias que deben reflejarse inmediatamente en toda la aplicación (como el tamaño de texto), se utiliza `ValueNotifier` combinado con `ValueListenableBuilder` a nivel de la raíz de la aplicación (`MyApp`), evitando la necesidad de un framework de gestión de estado de terceros para este caso de uso puntual.

6. Optimización de Navegación y Experiencia de Usuario

Se realizó una auditoría dedicada de experiencia de usuario sobre la aplicación en su estado actual, identificando hallazgos concretos y priorizándolos por impacto y esfuerzo de implementación. De esa auditoría, se implementaron efectivamente:

- Conexión de acciones previamente inertes en la interfaz (por ejemplo, el flujo de recuperación de contraseña, antes sin funcionalidad).
- Mejoras de retroalimentación visual en flujos de verificación (aviso explícito de "ambiente de prueba" en la pantalla de verificación de correo).

Se identificaron, pero no se implementaron en esta etapa, mejoras adicionales de la misma auditoría: retroalimentación háptica en acciones clave, diferenciación de errores de conectividad frente a otros errores genéricos, y estados de carga tipo *skeleton* en lugar de indicadores de progreso genéricos. Estas quedan documentadas como pendientes, no como trabajo completado.

Problemas Detectados

1. **Ausencia de un framework de gestión de estado** más allá de `StatefulWidget` y `ValueNotifier` puntual: viable en el tamaño actual del proyecto, pero puede generar mayor verbosidad y acoplamiento conforme crezca el número de estados compartidos entre pantallas distantes en el árbol de widgets.
2. **La auditoría de UX generó más hallazgos de los que se implementaron:** persiste una brecha documentada entre lo identificado y lo efectivamente construido (ver sección 6).
3. **No se pudo verificar la fidelidad de la implementación respecto a maquetas de diseño** (Figma), al no haberse tenido acceso al contenido visual del archivo de diseño durante la elaboración de este documento.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto en mantenibilidad a mediano plazo**, no un problema actual: el patrón funciona adecuadamente para el tamaño presente del proyecto.
- El riesgo del punto 2 es de **impacto en experiencia de usuario no crítico**: las mejoras pendientes son de naturaleza incremental (retroalimentación, percepción de velocidad), no defectos funcionales.
- El riesgo del punto 3 impide **certificar formalmente la correspondencia diseño-implementación** en este documento; se recomienda una validación específica una vez se tenga acceso al material de diseño.

Fortalezas Identificadas

- **Patrón de servicio-singleton aplicado de forma consistente** en la totalidad del proyecto, facilitando la localización de lógica de negocio por dominio.

- **Adaptación de pantallas por rol sin duplicación**, evitando múltiples versiones de una misma pantalla de detalle.
- **Extracción disciplinada de componentes compartidos** cuando se detecta duplicación real (caso `PostsGrid`), en lugar de abstracción prematura generalizada.
- **Persistencia de preferencias no ligadas a la cuenta** correctamente separada del ciclo de vida de la sesión de autenticación.

Áreas de Mejora

- Completar la implementación de los hallazgos de UX pendientes de la auditoría ya realizada.
- Evaluar la necesidad de un framework de gestión de estado ante el crecimiento futuro del número de pantallas y estados compartidos.
- Realizar una validación formal de fidelidad visual respecto a las maquetas de diseño en cuanto se tenga acceso al material correspondiente.

Recomendaciones

1. Priorizar la implementación de los hallazgos de UX pendientes de menor esfuerzo (retroalimentación háptica, diferenciación de errores de conectividad), dado que ya fueron identificados y solo falta ejecutarlos.
2. Mantener el patrón de servicio-singleton como estándar obligatorio para cualquier nuevo dominio funcional que se incorpore al frontend.
3. Programar una sesión de validación de fidelidad diseño-implementación tan pronto se disponga de acceso al archivo de Figma.

Conclusión

La estructura del frontend de HeartCoin refleja una organización disciplinada y consistente, apoyada en un patrón de servicio-singleton aplicado de forma uniforme y en una adaptación de pantallas por rol que evita la duplicación de código. El manejo de sesión delega correctamente en el SDK de Supabase, reservando mecanismos propios únicamente para el estado que no pertenece al ciclo de vida de la cuenta. Las áreas de mejora identificadas son de naturaleza incremental, con la salvedad de la validación de fidelidad de diseño, que permanece pendiente por falta de acceso al material visual de referencia.